NORMALISED-LEAST-MEAN-SQUARED ACOUSTIC AUDIO CANCELLATION

Muhammad Mekaill Zaffar Murtaza

28100176
LUMS Department of Electrical Engineering

Taha Siddique

27100217
LUMS Department of Electrical Engineering

ABSTRACT

In modern audio communications, such as video conferences or phone calls, the microphone picks up both the spoken audio, as well as echoes from across the room. To solve this, a well known approach of using the normalised-least-mean squared (NLMS) algorithm was used to learn the acoustic audio path and cancel it from the signal, returning a relatively clean signal with no echo. To address common challenges in real audio communications, such as double talk, or near-end-speech only, a basic Geigel detector was employed as well. This was done using Python on common consumer level hardware (HP-Victus and Macbook M4) across different environments and conditions. The result was a dependable echo cancellation, but with a highly constrained effectiveness that depended on the environment and conditions. It was concluded that NLMS alone is not reliable for practical use, with further research and enhancement needed to improve its effectiveness in different environments.

Index Terms— Acoustic Echo Cancellation (AEC), Normalized Least Mean Squares (NLMS), Adaptive FIR Filter, Echo Return Loss Enhancement (ERLE), Double Talk, Room Impulse Response.

1. INTRODUCTION

Acoustic echo is a persistent problem in hands-free communication systems such as video conferencing, mobile calls, and smart speakers. When a loudspeaker plays the far-end signal, it propagates through the room, reflects off walls and surfaces, and re-enters the microphone as a delayed, distorted version of the original. This echo is audible to the remote listener and significantly degrades speech quality and intelligibility. The problem was first formally addressed by Sondhi in 1967 [1], who proposed synthesising a replica of the echo and subtracting it from the microphone signal, establishing the foundation for modern acoustic echo cancellation (AEC). The core challenge is that the room impulse response governing the echo path is unknown, time-varying, and can span hundreds of milliseconds in reverberant environments [2]. Fixed filters are therefore inadequate, since they cannot track changes in the acoustic path caused by movement or environmental shifts. Adaptive filtering addresses this by continuously estimating the echo path from the reference signal. Among adaptive algorithms, the Normalized Least Mean Squares (NLMS) algorithm is widely used in AEC due to its computational simplicity, stability across varying input power levels, and ease of implementation [2, 3]. More advanced approaches such as frequency-domain adaptive filters [4] and variable step-size methods [5] have been proposed to improve convergence speed and tracking, but NLMS remains the standard baseline. A further complication arises during double-talk, when both the near-end and far-end speakers are simultaneously active. Uncorrelated near-end speech causes the adaptive filter to diverge if adaptation is not suspended [6]. Double-talk detectors (DTDs) are therefore a standard component of practical AEC systems, with the Geigel detector being one of the simplest and most commonly used approaches. This paper presents a complete NLMS-based AEC system implemented in Python with AI assistance, and evaluated on consumer hardware across four real-time conditions. The remainder covers theory (Section 2), system design (Section 3), experimental setup (Section 4), results (Section 5), and conclusion (Section 6).

2. THEORETICAL BACKGROUND

2.1. Signal Model

In a hands-free communication system, the far-end signal $x(n)$ is played through the loudspeaker. It travels across the room, reflects off surfaces, and arrives at the microphone as an echo. This echo is modelled as the convolution of $x(n)$ with the room impulse response $h(n)$:

$$y_{\text{echo}}(n) = \sum_{k=0}^{L-1} h(k)\, x(n-k) \tag{1}$$

where L is the length of the room impulse response. The microphone then picks up this echo alongside the near-end speech $s(n)$ and background noise $v(n)$:

$$d(n) = y_{\text{echo}}(n) + s(n) + v(n) \tag{2}$$

The goal of AEC is to estimate and subtract only $y_{\text{echo}}(n)$ from $d(n)$, leaving behind $s(n)$ and $v(n)$.

2.2. Adaptive FIR Filter

Since $h(n)$ is unknown, it is estimated using an adaptive FIR filter with N coefficients, stored in a vector:

$$\mathbf{w}(n) = [w_0(n),\ w_1(n),\ \ldots,\ w_{N-1}(n)]^T \tag{3}$$

The most recent N samples of the far-end signal are stored in a buffer:

$$\mathbf{x}(n) = [x(n),\ x(n{-}1),\ \ldots,\ x(n{-}N{+}1)]^T \tag{4}$$

The estimated echo is then the dot product of these two vectors:

$$\hat{y}_{\text{echo}}(n) = \mathbf{w}^T(n)\,\mathbf{x}(n) \tag{5}$$

The error signal, which is also the AEC output, is:

$$e(n) = d(n) - \hat{y}_{\text{echo}}(n) \tag{6}$$

When the filter has learned the echo path well, $\hat{y}_{\text{echo}}(n) \approx y_{\text{echo}}(n)$, and $e(n)$ contains mostly near-end speech and noise.

2.3. NLMS Update Rule

The filter coefficients $\mathbf{w}(n)$ are updated at every sample step to minimise the squared error $e^2(n)$. The NLMS update rule is:

$$\mathbf{w}(n{+}1) = \mathbf{w}(n) + \frac{\mu}{\|\mathbf{x}(n)\|^2 + \delta}\, e(n)\,\mathbf{x}(n) \tag{7}$$

where:

- μ is the step size, controlling how aggressively the filter updates. Larger μ converges faster but is less stable; smaller μ is more stable but slower.
- $\|\mathbf{x}(n)\|^2$ is the energy of the reference signal buffer, used to normalise the update so convergence speed does not depend on signal power.
- δ is a small regularisation constant that prevents division by zero when the input is silent.

2.4. Double-Talk Detection

Double-talk occurs when both the near-end speaker and far-end signal are active simultaneously. If NLMS adaptation continues during double-talk, the near-end speech acts as an uncorrelated disturbance, causing the filter to diverge and destroying the learned echo estimate.

The Geigel detector [**?**] addresses this by comparing the microphone signal amplitude to the peak amplitude of the reference buffer:

$$|d(n)| \geq c \cdot \max_k\{|\mathbf{x}(n-k)|\} \tag{8}$$

where c is a tunable threshold. If this condition is met, double-talk or near-end-only speech is inferred, and the weight update is suspended for a fixed hold-off period. This prevents the filter from diverging while still allowing fast resumption once the near-end activity ends.

2.5. Performance Metric: ERLE

Echo Return Loss Enhancement (ERLE) measures how much echo power the canceller removes, in decibels:

$$\text{ERLE}_{\text{dB}} = 10\log_{10}\left(\frac{\sum_n |y_{\text{echo}}(n)|^2}{\sum_n |e(n)|^2 + \epsilon}\right) \tag{9}$$

where ϵ is a small constant to avoid numerical issues. Higher ERLE means more echo removed. An ERLE above 15 dB is considered acceptable; above 25 dB is excellent.

3. SYSTEM DESIGN AND IMPLEMENTATION

3.1. NLMS Filter

The NLMS algorithm was implemented from scratch in Python without any built-in adaptive filtering libraries. The filter maintains two length-N arrays: the coefficient vector $\mathbf{w}(n)$ and the reference signal buffer $\mathbf{x}(n)$, both initialised to zero. At every sample, the buffer shifts to insert the new far-end sample, the estimated echo is computed via dot product, the error is calculated, and the weights are updated according to (7) if adaptation is permitted.

The final parameters after tuning are given in Table 1.

Table 1. NLMS implementation parameters.

Parameter	Symbol	Value
Sample rate	f_s	16,000 Hz
Filter length	N	2,047 taps
Step size	μ	0.35
Regularisation	δ	10^{-5}
Block size	—	512 samples
Geigel threshold	c	1.0
Hold-off period	—	1,600 samples (100 ms)

The sample rate of 16 kHz captures all frequencies relevant to speech while keeping computational load manageable. A filter length of 2,047 taps was chosen because at 16 kHz this covers approximately 128 ms of room impulse response, which is long enough to model most significant reflections in a typical indoor space; shorter filters risk missing late reflections, while longer filters increase computational cost and slow convergence. The step size $\mu = 0.35$ was set conservatively within the recommended range of 0.3–0.7: a larger μ would converge faster but risk instability or steady-state noise, while a smaller μ would be overly slow for a real-time system. The regularisation constant $\delta = 10^{-5}$ sits in

the middle of the recommended range of 10^{-6} to 10^{-4}, large enough to prevent division by zero during silence but small enough not to artificially slow adaptation. A block size of 512 samples gives a per-block latency of 32 ms, staying well within the 100 ms real-time requirement. The hold-off period of 1,600 samples (100 ms) was chosen to be long enough to cover typical speech bursts without freezing adaptation for unnecessarily long periods.

3.2. Geigel Double-Talk Detector

The Geigel detector checks every incoming sample against condition (8). If the microphone signal exceeds the scaled peak of the reference buffer, the hold-off counter is reset to 1,600 samples (100 ms) and adaptation is frozen for that duration. This handles three practical conditions: far-end only (adaptation runs freely), near-end only (buffer peak is zero, so any non-zero mic signal triggers a freeze), and double-talk (mic signal exceeds the scaled far-end peak). The threshold $c = 1.0$ was used as the default, meaning the microphone signal must exceed the full peak of the reference buffer to trigger a freeze; a lower c would make the detector more sensitive but risk false freezes from loud room noise.

An optional calibration routine was also implemented: five seconds of far-end audio is played while the user remains silent, and the distribution of $|d(n)|/\max|\mathbf{x}(n)|$ ratios is recorded. The threshold c is then set to the 95th percentile of this distribution with a 30% safety margin, adapting the detector to actual room attenuation.

3.3. Real-Time Audio Pipeline

The real-time system uses PyAudio to open a full-duplex stream at 16 kHz with a block size of 512 samples, giving a processing latency of approximately 32 ms per block. The far-end signal is read from a WAV file and played through the laptop speakers, while the microphone simultaneously captures the acoustic echo and any near-end speech.

Because the loudspeaker-to-microphone path introduces a hardware delay not accounted for in the filter model, an auto-calibration step runs at startup. One second of white noise is played while the microphone records; the cross-correlation of the two signals identifies the peak delay, and a ring buffer of that length aligns the reference signal with the microphone signal before it enters the NLMS pipeline. Without this alignment, the filter would attempt to learn a misaligned echo path, significantly degrading cancellation performance.

The demonstration is structured into four sequential phases: baseline with adaptation disabled to confirm echo is present, convergence with the user silent to show the echo fading, double-talk where the user speaks while far-end plays, and path change where the acoustic environment is physically altered mid-session to demonstrate re-convergence.

4. EXPERIMENTAL SETUP AND METHODOLOGY

4.1. Hardware and Software

All experiments were conducted on an HP Victus laptop and a MacBook, using each device's built-in microphone and speakers. The system was implemented in Python using NumPy for numerical operations and PyAudio for real-time audio I/O. OS-level audio enhancements were disabled where possible to prevent interference with the AEC measurements. The far-end signal was a pre-recorded 50-second WAV file at 16 kHz.

4.2. Initial Simulation Study

Before performing practical, real-world tests, a series of simulated tests were run to ensure the algorithm functioned in ideal conditions. The most primitive of these tests were done with sinusoids. That is, the near end and far end signals were simulated with sinusoids, with added Gaussian noise to add the factor of noise.

$$x_n = \sin(2\pi \cdot 400 \cdot t) \tag{10}$$

Fig. 1. The sinusoidal equation used to define the far end sample.

Table 2. Results of basic sinusoid testing

#	N	Step Size	c	Halt	ERLE greater than 15dB
1	511	0.3	0.7	1600	0
2	511	0.3	0.65	1600	1
3	511	0.3	0.6	1600	1
4	511	0.7	0.65	1600	3
5	511	0.6	0.65	1600	3
6	511	0.5	0.65	1600	4
7	511	0.4	0.65	1600	3
8	1023	0.4	0.65	1600	5
9	1023	0.5	0.65	1600	2
10	2047	0.4	0.65	1600	2

The table shows tests for a series of values. For each test, the algorithm was run 10 times with the simulated far-end input. The number of times the maximum ERLE was within acceptable levels - 15dB - was noted. This was used to deduce an initial range of acceptable values, around which more robust testing could take place. These are shown in columns 6 and 8, which have the highest number of acceptable values.

4.3. Advanced Simulation Study

More advanced tests were run with additional sinusoids, as well as audio real-time audio samples. These samples were

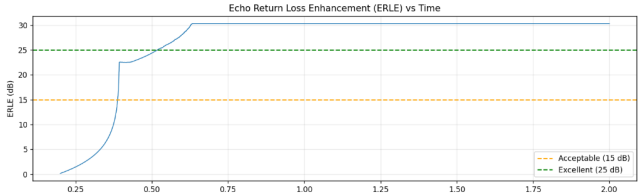

Fig. 2. Select best ERLE for basic sinusoid test: N = 1023, step size = 0.5, c = 0.65

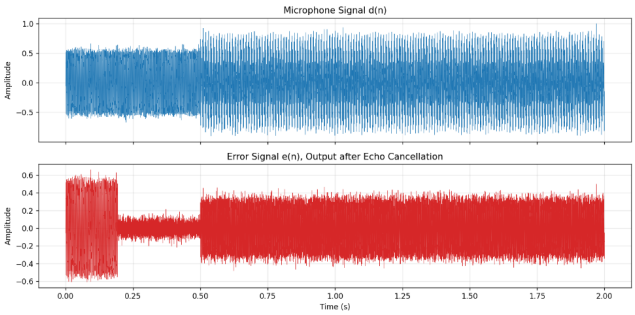

Fig. 3. Select best results for basic sinusoid test: N = 1023, step size = 0.5, c = 0.65

carefully selected from online training algorithm audio training data to ensure a suitable amount of noise, while keeping the sample relatively clean and continuous.

The initial tests with additional sinusoids were done using the following far end sample.

$$x_n = \frac{\sin(2\pi \cdot 300t) + \sin(2\pi \cdot 700t) + \sin(2\pi \cdot 1200t)}{3} \tag{11}$$

This provided a more robust and realistic sample to test with. However, it was realised after numerous tests that this sample did not provide additional practical benefits in testing, as compared to the basic sinusoid.

The tests used multiple metrics for evaluation. These included ERLE, filter misalignment, as well as the use of a sliding window.

This was thus followed by the use of audio samples in testing, to maintain ideal conditions, while creating more complex scenarios. A 28 second-long audio book sample from the LibreSpeech ASR Corpus was used, providing crisp audio for testing. An interesting point to note is that, in some tests, the filter misalignment showed a sudden rapid increase then remained stable. Moreover, in such tests the ERLE was actually negative, indicating that convergence had failed and the signal was being amplified. Additional testing allowed for improvements in the fine-tuning of the algorithm,

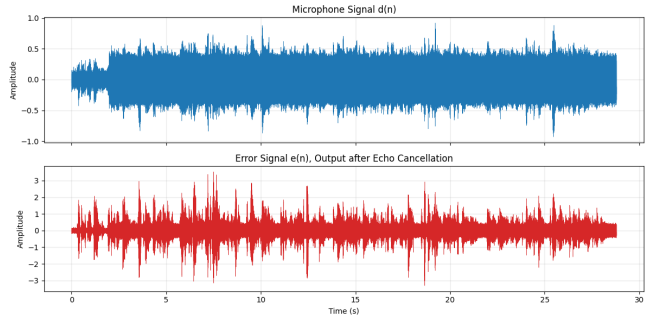

Fig. 4. Select test results for real audio sample simulation. Geigel threshold = 1, step size = 0.3

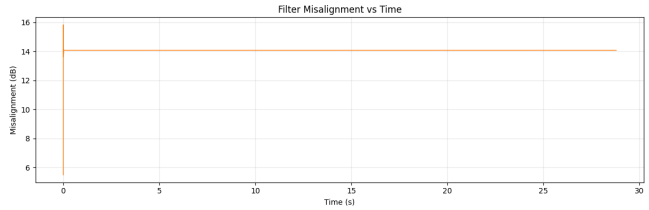

Fig. 5. Filter misalignment showing sharp initial increase

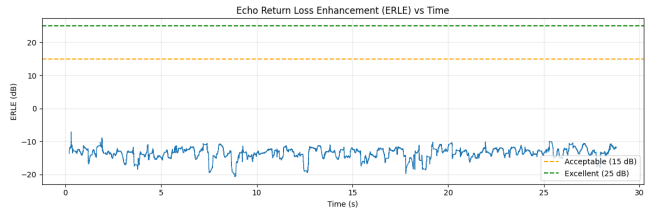

Fig. 6. Sample audio test showing negative ERLE

4.4. Real-Time Evaluation

The real-time pipeline opened a full-duplex PyAudio stream at 16 kHz with a block size of 512 samples. At startup, the auto-calibration routine played one second of low-amplitude white noise through the speakers while recording the microphone. The cross-correlation of the played and recorded signals identified the hardware delay. Due to variability across devices and room conditions, the calibration occasionally yielded unrealistic results, in which case the system defaulted to a fixed delay of 60 ms. A ring buffer of the detected delay length then aligned the reference signal with the microphone signal throughout the session.

The evaluation was structured into four sequential phases, each running for the full 50-second duration of the far-end audio file. In Phase 0, adaptation was disabled and the raw microphone output was recorded to confirm echo was present and OS-level cancellation was not active. In Phase 1, adaptation was enabled with the user silent to allow the filter to converge; performance was evaluated by plotting the microphone signal, error signal, and ERLE over time using a 50 ms sliding window. In Phase 2, the user spoke naturally while the far-end signal played to evaluate double-talk handling, with ERLE tracked using a 300 ms sliding window; Geigel detector activity was additionally plotted as the fraction of samples per block for which adaptation was frozen, ranging from 0 (fully adapting) to 1 (fully frozen), allowing visual identification of when the detector triggered. In Phase 3, the acoustic environment was physically altered mid-session by placing objects such as books and metal water bottles in front of the speaker to test re-convergence after a path change. All testing was conducted in a typical bedroom environment. Phase outputs were also recorded to WAV files and evaluated through listening tests to assess audible echo reduction.

5. RESULTS AND DISCUSSION

5.1. Phase 1: Convergence

Figure 1 shows the microphone signal, error signal, and ERLE over the 50-second convergence phase. The microphone signal had an amplitude of approximately ± 0.025, which the filter reduced to roughly ± 0.005 in the error signal, a visible and audible reduction. The ERLE plot fluctuated between 0 and 20 dB throughout, averaging around 8–12 dB, with occasional peaks near 20 dB. It rarely crossed the 15 dB acceptability threshold consistently.

This is lower than expected and can be attributed to a few factors. The bedroom environment introduces irregular reflections that a linear FIR filter cannot fully model. The hardware delay calibration, which defaulted to 60 ms on some devices, may have been imprecise, causing slight misalignment between the reference and microphone signals. Additionally, the ERLE computation uses a 50 ms sliding window, which is sensitive to short-term fluctuations and can produce noisy readings even when cancellation is working well. The echo was nonetheless audibly reduced, which the output WAV file confirmed.

5.2. Phase 2: Double-Talk

Figure 2 shows the Geigel DTD activity during Phase 2. With the threshold manually tuned, the detector correctly identified approximately 6 distinct speech bursts across the recording, freezing adaptation during each. Between speech bursts, the fraction frozen returned to zero, confirming that adaptation resumed normally in silence.

The original threshold of $c = 1.0$ produced no detections at all. This is because the room attenuates the echo significantly before it reaches the microphone, so $|d(n)|$ never approached $\max|\mathbf{x}(n)|$ at full scale. The Geigel detector as formulated assumes the microphone amplitude can approach loudspeaker amplitude, which is unrealistic in a typical room without calibration. Lowering c to hardware-dependent values was a manual workaround; ideally the calibration routine would determine this value automatically from the actual room attenuation.

Some residual echo was still audible during sustained loud speech, suggesting the detector occasionally under-detected double-talk. This is a known limitation of the single-threshold Geigel architecture, which compares instantaneous amplitudes rather than modelling the statistical relationship between $x(n)$ and $d(n)$ [6].

5.3. Phase 3: Acoustic Path Change

Figure 3 shows the microphone and error signals during Phase 3. The error signal amplitude is noticeably larger in the first few seconds, corresponding to the period immediately after the acoustic path was altered by placing books and metal water bottles in front of the speaker. The previously learned filter coefficients no longer matched the new echo path, causing a brief but audible echo burst. The error signal then gradually settled, with amplitude reducing after approximately 8–10 seconds as the filter re-converged to the new path.

This behaviour is expected and confirms that the NLMS algorithm can track a changing acoustic environment. The re-convergence time depends on the step size μ: a larger μ would have recovered faster but at the cost of greater steady-state noise, while the chosen value of 0.35 struck a reasonable balance.

5.4. General Limitations

The most consistent limitation across all phases was the gap between real-room and ideal performance. The filter length of 2,047 taps covers 128 ms of impulse response, but real rooms produce overlapping reflections with non-linear behaviour that a linear FIR model cannot capture. Post-filtering

or frequency-domain processing would be needed to close this gap.

Hardware variability was also a practical issue. The system was tested on both an HP Victus and a MacBook, and the auto-calibration produced different delay estimates on each device, sometimes defaulting to the 60 ms fallback. This inconsistency affected cancellation quality across sessions and made it difficult to reproduce identical results.

Finally, the Geigel threshold required manual tuning to function in a real room, which limits the system's plug-and-play usability. A more robust solution would use the calibration routine to set c automatically before each session.

6. CONCLUSION

This paper presented a Python-based NLMS acoustic echo canceller evaluated on consumer hardware across four real-time phases. The system successfully demonstrated audible echo reduction during convergence, reasonable double-talk handling with a manually tuned Geigel detector, and re-convergence after a physical change to the acoustic environment.

Performance in practice fell short of ideal. Real-room ERLE averaged 8–12 dB, below the 15 dB acceptability threshold, due to irregular room reflections, hardware delay estimation errors, and the inherent ceiling of a linear FIR model. The Geigel detector required hardware-dependent manual tuning to trigger correctly, as the default threshold of $c = 1.0$ was too high for typical room attenuation levels.

These results confirm that NLMS alone is not sufficient for robust real-world echo cancellation. Practical improvements would include frequency-domain adaptive filtering for faster convergence, a statistical double-talk detector based on cross-correlation rather than amplitude thresholding, and a residual echo suppression stage to handle non-linear reflections that the adaptive filter cannot model. Despite these limitations, the system met the core objectives of the project and provided a clear demonstration of adaptive filtering concepts in a real acoustic environment.

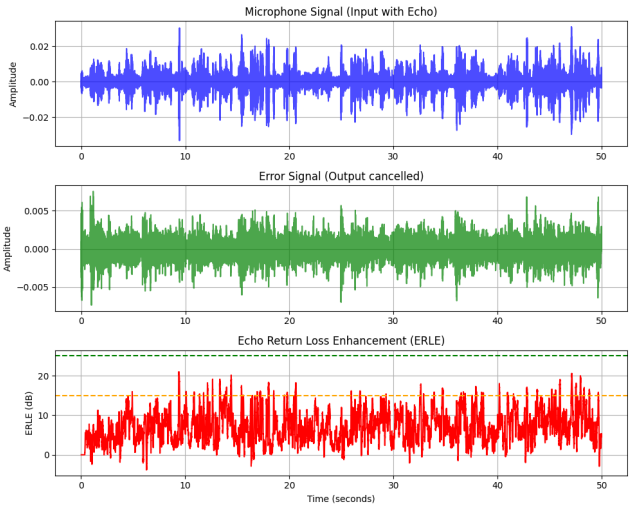

Fig. 7. Phase 1: Microphone signal, error signal, and ERLE over time.

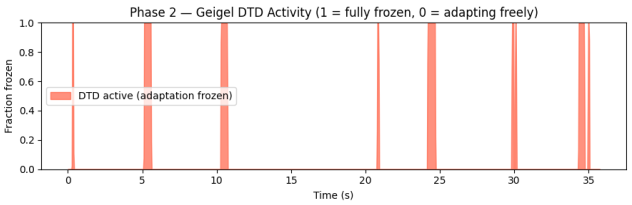

Fig. 8. Phase 2: Geigel DTD activity.

7. REFERENCES

[1] M. M. Sondhi, "An adaptive echo canceller," *Bell System Technical Journal*, vol. 46, no. 3, pp. 497–511, 1967.

[2] S. Haykin, "Adaptive filter theory," *Prentice Hall*, 2002.

[3] D. L. Duttweiler, "Proportionate normalized least-mean-squares adaptation in echo cancellers," *IEEE Transactions on Speech and Audio Processing*, vol. 8, no. 5, pp. 508–518, 2000.

[4] J.-M. Valin, "On adjusting the learning rate in frequency domain echo cancellation with double-talk," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 15, no. 3, pp. 1030–1034, 2007.

[5] S. Makino, Y. Kaneda, and N. Koizumi, "Exponentially weighted stepsize NLMS adaptive filter based on the statistics of a room impulse response," *IEEE Transactions on Speech and Audio Processing*, vol. 1, no. 1, pp. 101–108, 1993.

[6] J. Benesty, D. R. Morgan, and J. H. Cho, "A new class of doubletalk detectors based on cross-correlation," *IEEE*

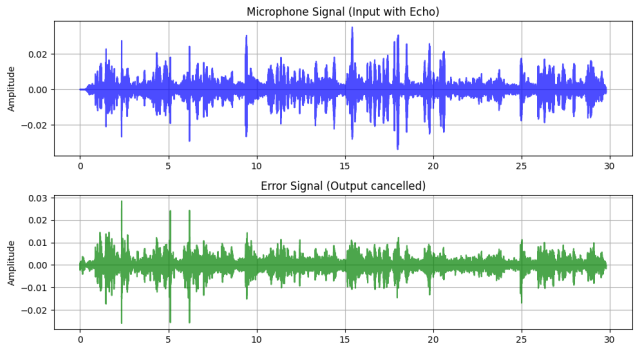

Fig. 9. Phase 3: Microphone and error signals after acoustic path change.

Transactions on Speech and Audio Processing, vol. 8, no. 2, pp. 168–172, 2000.